# Movie Ticketing System

# Software Requirements Specification

<Version 2.0>

<03/05/26>

Group #19
Michelle Khaliqi, Nandana Manu,
Katarina Cedolin

Prepared for
CS 250- Introduction to Software Systems
Instructor: Dr. Gus Hanna
Spring 2026

# LINKS:

Github Repository: https://github.com/nmanu72/CS250SystemDesignProject

Test Case Document:
https://docs.google.com/spreadsheets/d/1pTYvXyRgL1WFtNuxRrWvFCQTEsEf0DMt/edit?usp=sharing&ouid=105342803146073255555&rtpof=true&sd=true

# Revision History

| Date | Description | Author | Comments |
|---|---|---|---|
| 2/15 | Version 1.0 | Group 19 | < SRS 1.0 > |
| 2/19 | Update Version 1.0 | Group 19 | < SRS 1.0 > |
| 3/05 | Version 2.0 | Group 19 | <SRS 2.0> |
| 3/19 | Version 3.0 | Group 19 | <SRS 3.0> |

# Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

| Signature | Printed Name | Title | Date |
|---|---|---|---|
| | <Your Name> | Software Eng. | |
| | Dr. Gus Hanna | Instructor, CS 250 | |
| | | | |

# Table of Contents

**Revision History** ..... ii

**Document Approval** ..... ii

**1. Introduction** ..... 1

1.1 Purpose ..... 1
1.2 Scope ..... 1
1.3 Definitions, Acronyms, and Abbreviations ..... 1
1.4 References ..... 2
1.5 Overview ..... 2

**2. General Description** ..... 2

2.1 Product Perspective ..... 2
2.2 Product Functions ..... 3
2.3 User Characteristics ..... 3
2.4 General Constraints ..... 4
2.5 Assumptions and Dependencies ..... 4

**3. Specific Requirements** ..... 4

3.1 External Interface Requirements ..... 5
*3.1.1 User Interfaces* ..... 5
*3.1.2 Hardware Interfaces* ..... 5
*3.1.3 Software Interfaces* ..... 6
*3.1.4 Communications Interfaces* ..... 6
3.2 Functional Requirements ..... 6
*3.2.1 <Concurrent User Support>* ..... 6
*3.2.2 <Web Browser Access>* ..... 7
*3.2.3 <Bot Detection and Protection>* ..... 7
*3.2.4 <Database Interface Integration>* ..... 8
*3.2.5 <Ticket Purchase Constraints>* ..... 8
*3.2.6 <Administrator Mode Support>* ..... 9
*3.2.7 <Customer Feedback System>* ..... 10
*3.2.8 <Discount Ticket Support>* ..... 10
*3.2.9 <Online Review Scraping + Integration>* ..... 10
*3.2.10 <Multilingual Support>* ..... 11
*3.2.11 <Loyalty Account Management>* ..... 11
3.3 Use Cases ..... 12
*3.3.1 Use Case #1* ..... 12
*3.3.2 Use Case #2* ..... 12
*3.3.3 Use Case #3* ..... 13
3.4 Classes / Objects ..... 14
*3.4.1 <User>* ..... 14
*3.4.2 <Ticket>* ..... 14
*3.4.3 <Movie>* ..... 15
*3.4.4 <Administrator>* ..... 15
*3.4.5 <Payment>* ..... 16
*3.4.6 <Feedback>* ..... 16
*3.4.7 <Theatre>* ..... 16
3.5 Non-Functional Requirements ..... 17
*3.5.1 Performance* ..... 17
*3.5.2 Reliability* ..... 17
*3.5.3 Availability* ..... 17
*3.5.4 Security* ..... 17
*3.5.5 Maintainability* ..... 17
*3.5.6 Portability* ..... 18

**4. Software Design Specification** ..... 18

4.1 Software Architecture Overview ..... 18

4.2 UML Class Diagram..... 19
4.3 Software Architecture Diagram..... 19
4.4 Software Architecture Description..... 19
*4.4.1 Interfaces..... 20*
*4.4.2 Databases..... 20*
*4.4.3 External Services..... 21*
*4.4.4 Data Management Strategy..... 21*
*4.4.5 Tradeoff Discussion..... 21*
4.5 Development Plan and Timeline..... 23
*4.5.1 Partitioning of Tasks..... 21-23*
*4.5.2 Team Responsibilities..... 23-24*
4.6 Test Plan..... 25-27

# 1. Introduction

## 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a clear outline of the requirements for building a Movie Ticketing System, including the system's functionality and constraints. This document is intended for the developers responsible for creating the Movie Ticketing System.

## 1.2 Scope

The Movie Ticketing System is to be accessed by customers and primary users of the system, which include system administrators, cinema staff, and movie theater customers.

The Movie Ticketing System is a software application designed for customers to browse through available movies, select designated showtimes, reserve seats, and purchase available tickets online. After confirmation and payment, customers will receive an email that includes their receipt and a personal QR code that is ready to be scanned by cinema staff in person. Additionally, the system will allow administrators to update movie listings, showtimes, and pricing information. Cinema staff will confirm bookings, view prices, and monitor showtimes without access to modify data. The Movie Ticketing System will not stream movies or provide services for snacks.

The goal of this Movie Ticketing System is to be a faster and more convenient method of purchasing movie tickets, preventing long lines of customers who purchase tickets in person. The system aims to handle high user traffic hours of purchases, update seat availability so the same seat is not purchased by two individuals, and securely process online payments. The constraints for this section includes our deadline of May 7, 2026 for the document.

## 1.3 Definitions, Acronyms, and Abbreviations

The following terms, acronyms, and abbreviations are used in this SRS and are defined here to support consistent interpretation of the requirements.

API: Application Programming Interface
CAPTCHA: Completely Automated Public Turing test to tell Computers and Humans Apart
HTTP: Hypertext Transfer Protocol
HTTPS: Hypertext Transfer Protocol Secure
HTTP(S): HTTP or HTTPS
IMDB: Internet Movie Database
MTBF: Mean Time Between Failures
QR: Quick Response
SRS: Software Requirements Specification

## 1.4 References

The following sources were used in preparation of the SRS:

Cornell University. (2019, Spring). *Use cases* [Lecture slides]. Cornell University, Department of Computer Science. https://www.cs.cornell.edu/courses/cs5150/2019sp/slides/7-use-cases.pdf

GeeksforGeeks. (2026). *Unified Modeling Language (UML) class diagrams*. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/

KrazyTech. (2025). *Sample software requirements specification (SRS) report – Airline database*. KrazyTech. https://krazytech.com/projects/sample-software-requirements-specificationsrs-report-airline-database

McMillan, J. *Requirements handout*. Clemson University, Department of Computing. https://people.computing.clemson.edu/~johnmc/courses/cpsc371/RequirementsHandout.pdf

Paulk, M. C. (2019, May 20). *An introduction to software architecture and design I* [PDF]. The University of Texas at Dallas. https://personal.utdallas.edu/~mcp130030/talks/t20190520arch.pdf

Relevant Software. (2023). *Software requirements specification (SRS) document*. Relevant Software Blog. https://relevant.software/blog/software-requirements-specification-srs-document/

## 1.5 Overview

This Software Requirements Specification document outlines all the features and considerations necessary in developing the Movie Ticketing System. Section 2 provides a general description of the system, including the main functions, user characteristics, constraints, and assumptions. Section 3 lists the specific requirements, such as how users interact with the system, specific features of the system, and use cases. This SRS is organized to help the software developers understand the system and use the requirements listed to design and implement the software.

# 2. General Description

## 2.1 Product Perspective

The Movie Theater Ticketing System is a web-based software product that supports the end-to-end process of browsing movies, selecting showtimes, reserving tickets, purchasing tickets, and verifying tickets in person. The system complements in-person ticket sales, box office sales, and online purchases by ensuring all purchases reflect the same seat/ticket inventory in real time to prevent conflicts and double-booking. The initial deployment targets 20 theaters in the San Diego area operating in the Pacific Time zone with a shared, centrally managed database partitioned by theater.

The system interacts with several external elements; customers, through a web interface, cinema staff, through ticket validation, administrators, through management functions for listings, schedules and pricing, and supporting services such as payment processing and email delivery for receipts and tickets. The theater network includes regular-auditorium theaters (general admission) and deluxe-auditorium theaters (reserved/assigned seating); seat inventory rules differ by auditorium type.

## 2.2 Product Functions

The proposed system will provide the following major functionalities:

- Movie discovery and browsing: Display to customer current and upcoming movies, descriptions, ratings, and showtimes by theater location.
- Showtime and seat selection: Allow users to choose a theater, date/time, and available seats from a seating map.
- Ticket purchase: Allow users to purchase tickets of several categories (Child, Adult, Senior, Discount if applicable) and reserve these seats with a checkout flow.
- Transaction confirmation: Provide a confirmation page and send a digital (email or text) receipt containing purchase details and a unique ticket number/barcode.
- Ticket validation: Allow cinema staff to confirm ticket validity and view showtime and seat details without editing listings.
- Administration and maintenance: Allow administrators to manage movies, showtimes, seat inventory, and pricing.
- Possible data collection for activity logs: Track and summarize sale counts and activity logs for administrative oversight.

## 2.3 User Characteristics

The system is expected to be used by three primary user groups, whose characteristics influence usability, accessibility, and interface design.

- Customers (general public): Vary widely in technical abilities. Will likely require a simple, mobile-friendly web interface with clear navigation, minimal steps to purchase, and understandable error messages. First-time users should not require instructions to carry out their intended actions. Large numbers of users at a time should be expected.

Customers may purchase from mobile or desktop browsers; flows should remain simple under high concurrency.
- Cinema Staff: Frequent users during operating hours who prioritize speed and accuracy when verifying tickets and resolving entry issues. Training time is limited so workflows should be efficient and minimize common errors. Staff also handle exceptions such as in-person refunds and entry issues, so validation workflows should surface clear ticket status and show details quickly.
- System Administrators: Technically comfortable users who manage listings and operational data and are very familiar with the non-technical operations of the movie theater. They need dependable configuration tools, clear visibility into changes, and the ability to quickly correct items such as showtime schedules and pricing. Admins may require override tools to correct customer purchase mistakes and adjust showtimes/theaters with minimal disruption.

## 2.4 General Constraints

The following constraints may limit design and implementation options.
- Browser-only access: The system is intended to run in modern web browsers and should not require users to install an application.
- Security and privacy considerations: The system processes sensitive user information and must incorporate the appropriate protections.
- Operational environment: The system must function during peak demand and be able to accommodate growing crowds. It must continue to maintain accurate seat availability to reduce double-booking.
- External services: Payment processing and email delivery will depend on third-party services.
- Schedule constraints: Delivery of the project milestones are bounded by course deadlines. (4 months)
- Budget constraints: Budget is limited to $500,000.

## 2.5 Assumptions and Dependencies

The SRS assumes the following conditions will hold; changes to these assumptions may require updating system requirements.
- Reliable internet access is available for customers and theater staff.
- Email delivery service is available and configured to send confirmations to customers after purchases.
- Payment processing service is available for online transactions.
- A centralized database is available to store movies, showtimes, seat inventory, users, transactions, and administrative data.
- Theater operational data is appropriately maintained by administrators.
- Ticket verification capabilities exists for staff, eg. a scanner or device capable of validating bookings.

# 3. Specific Requirements

## 3.1 External Interface Requirements

### 3.1.1 User Interfaces

- The system shall provide a web-based graphical user interface accessible through modern web browsers.
- The interface shall allow users to browse movies, select showtimes, choose seats (deluxe theaters only), and complete purchases with no more than 5 steps.
- The interface shall clearly display:
    - Movie title and description.
    - Theatre location.
    - Showtime.
    - Ticket prices (including applied discounts, if applied).
    - Seat availability with real time updates.
- The system shall provide a visible 5-min timer during checkout and ticket selection.
- The system shall provide clear confirmation messages for successful actions and clear error messages for issues.
- Error messages must be clear and descriptive, guiding users on how to fix the issues.
- The user interface shall support the following languages: English, Spanish, Swedish.
- The interface shall be usable on both desktop and mobile web browsers without requiring the download of an application.
- The user interface shall be simple to navigate.
- User interface shall not require the user to perform difficult tasks or actions, and should allow a first-time user to navigate the system without needing to be directed.

### 3.1.2 Hardware Interfaces

- The system shall operate on any device capable of running modern web browsers (ie. desktop computers, laptops, tablets, smart phones).
- Cinema staff shall use QR code scanners to validate digital tickets.
- The system shall support the printing of physical tickets at box office kiosks or at home to any device with a connected printer.

### 3.1.3 Software Interfaces

- The system shall interface with a centralized database which stores:
    - Movie listings
    - Showtimes
    - Seat availability
    - User accounts
    - Transaction history
    - System logs
    - Error logs
- The system shall work with external payment processing services which handle at least the following: Credit Card, PayPal, Bitcoin transactions.

- The system shall work with an email service provider to deliver digital tickets, receipts and confirmation messages.
- The system shall work with external review API services to retrieve movie ratings and critic quotes (IMDB, Rotten Tomatoes, etc.).

### 3.1.4 Communications Interfaces

- All communication between the client and server shall occur through HTTPS.
- The system shall support secure HTTP requests for user login, sign up, ticket purchasing, administrative requests, etc.
- The system shall send email confirmations through a secure email method.
- The system shall maintain session management through secure use of cookies.

## 3.2 Functional Requirements

*This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.*

### 3.2.1 <Concurrent User Support>

3.2.1.1 Introduction

This feature ensures that the system can handle at least 1000 users simultaneously without system failure or degradation of performance. This feature is important when it comes to high traffic hours, such as during the release of a popular movie or on weekend evenings.

3.2.1.2 Inputs

- Login Requests
- Ticket selection/purchase requests
- Movie browsing requests

3.2.1.3 Processing

- The system shall manage multiple connections concurrently to the server.
- The system shall maintain session integrity for all concurrent users.
- The system shall implement a queue mechanism for high-demand showings to regulate user access and prevent server overload.

3.2.1.4 Outputs

The system shall provide:

- Real-time seat availability for all users.
- Confirmation of successful login.
- Updated ticket availability reflected online.

3.2.1.5 Error Handling

The system shall:

- Display an error message if the maximum concurrent user threshold is exceeded.
- Prevent duplicate ticket purchases made by simultaneous requests.
- Relist incomplete/failed transactions immediately.
- Log all failures for administrator review.

### 3.2.2 <Web Browser Access>

3.2.2.1 Introduction

The system shall operate fully as a web browser application and shall not require installation of a mobile or desktop application.

3.2.2.2 Inputs
- HTTP(S) requests from user browsers
- User form submissions (login, purchases, feedback)
- Session cookies

3.2.2.3 Processing
The system shall:
- Render dynamic web pages through browser-compatible technologies.
- Ensure compatibility with modern browsers (Chrome, Safari, Edge, etc.)
- Maintain secure user sessions.

3.2.2.4 Outputs
- Interactive web interface.
- Secure transaction confirmation screens.

3.2.2.5 Error Handling
- Display error message if browser is not supported.
- Display connection error if server is unresponsive/unreachable.
- Log errors for administrator oversight.

### 3.2.3 <Bot Detection and Protection>

3.2.3.1 Introduction
The system shall detect, then protect from automated bot activity which is intended to mass purchase tickets to high-demand movies. Bots buy tickets in mass numbers and resell them, prevention protects real customers.

3.2.3.2 Inputs
- Login attempts.
- Purchase frequency data.
- User interaction timing data.
- CAPTCHA verification results.
- Attempts to purchase tickets in massive quantities.

3.2.3.3 Processing
- Require CAPTCHA verification at both login and checkout.
- Monitor abnormal requests and frequencies (ie. bulk purchases, excessive rapidity).
- Flag suspicious behavior patterns.
- Suspend accounts temporarily which exhibit bot-like behavior and request identity verification (ie. CAPTCHA) to regain access.

3.2.3.4 Outputs
- CAPTCHA trials.
- Account suspension notifications with steps to unsuspend.
- Alerts to administration.
- Log of bot activity.

3.2.3.5 Error Handling
- Deny transactions in case of failed CAPTCHA.
- Deny login requests in case of failed CAPTCHA.
- Log suspicious behavior.

### 3.2.4 <Database Interface Integration>

3.2.4.1 Introduction

The system shall interface with a centralized database that manages the movies, theaters, showtimes, ticket inventories, users, transactions, and administrator data.

3.2.4.2 Inputs

- Showtime lookup requests.
- Ticket purchase requests.
- Login requests (user and administrator side).
- Feedback submissions.

3.2.4.3 Processing

- Retrieve showtime and seat availability data from database.
- Retrieve ticket counts and pricing information.
- Immediately update ticket counts after purchase is successfully completed.
- Divide data (seats, showtimes, prices, etc.) by theater location.
- Store transaction history and user activity.
- The system shall maintain system-wide daily logs of all ticket purchases.

3.2.4.4 Outputs

- Updated seat availability.
- Updated ticket availability.
- Updated showtime information on website.
- Updated + accurate statistics of ticket sales for administrators.
- Transaction history and receipts available for users.

3.2.4.5 Error Handling

- Return transactions if database update fails, issue immediate refund.
- Display errors in case the database is unavailable.
- Log database failures.

### 3.2.5 <Ticket Purchase Constraints>

3.2.5.1 Introduction

The system shall enforce ticket quantity and time restrictions. The user can purchase no more than 20 tickets at a time, and the tickets shall only be available from two weeks prior to showtime and 10 minutes after the showtime begins.

3.2.5.2 Inputs

- Selected showtime.
- Number of tickets requested.
- Current system time and date.
- Payment information (credit card, PayPal, BitCoin, etc.)

3.2.5.3 Processing

- Validate that purchase does not exceed 20 tickets per transaction.
- Validate that showtime is within the two week booking window.
- Validate that purchase is not made later than 10 min. after showtime start.
- Start a checkout timer with (~5 min) upon ticket selection.
- Release reserved tickets upon timer expiration.
- Tickets may be returned or exchanged prior to showtime only if purchased through a registered user account.

- For deluxe theatres, the system shall allow users to select and reserve specific seat numbers.
- For regular theatres, the system shall track ticket quantity without assigned seating.

3.2.5.4 Outputs
- Purchase confirmations.
- Countdown timer display.
- Validation success or failure message.

3.2.5.5 Error Handling
- Display message if ticket limit exceeded.
- Display message if ticket booked outside valid window.
- Deny refund/exchange requests made after showtime begins.
- Cancel transaction when timer expires.
- Display message when timer expires.

### 3.2.6 <Administrator Mode Support>

3.2.6.1 Introduction
The system shall provide administrator controls and support for system management.

3.2.6.2 Inputs
- Administrator login requests.
- Administrator login credentials.
- Ticket override requests.
- User account management actions.
- Showtime update requests.
- Price update requests.
- Movie updates and overrides.

3.2.6.3 Processing
- Authentication of administrator credentials.
- Update movie listings, showtimes, prices, etc.
- Override of user purchase when necessary.
- Ability to block/unblock user accounts.
- Ability to temporarily suspend/unsuspend user accounts.

3.2.6.4 Outputs
- Updated system data.
- Sales reports for administration.
- Confirmation messages for successful admin actions.

3.2.6.5 Error Handling
- Deny access if authentication fails.
- Log unauthorized admin access attempts.
- Device and login verification tools to prevent unauthorized admin access.

### 3.2.7 <Customer Feedback System>

3.2.7.1 Introduction
The system shall collect and store customer feedback after ticket purchase.

3.2.7.2 Inputs
- User feedback rating (Smiley selection/Numerical 1-10).

- Optional text comments.

3.2.7.3 Processing
- Validate feedback submission.
- Store feedback in the database linked to the transaction and log.
- Rating aggregate values for administration report.

3.2.7.4 Outputs
- Confirmation of feedback submission.
- Thank you message post feedback submission.
- Updated feedback analysis and data for administrators.

3.2.7.5 Error Handling
- Display error message if feedback submission fails.
- Prevent duplicate feedback submissions for the same transaction.

### 3.2.8 <Discount Ticket Support>

3.2.8.1 Introduction
The system shall support discounted tickets and categories.

3.2.8.2 Inputs
- Selected discount type (ie. student, military, senior, etc.)
- Proof of eligibility verification.
- Promotional code validation.

3.2.8.3 Processing
- Validate purchase request.
- Validate discount eligibility.
- Apply fixed/percentage price reduction based on discount.
- Recalculate total cost.

3.2.8.4 Outputs
- Updated discount ticket price
- Confirmation of discount application.

3.2.8.5 Error Handling
- Display message if ineligible or eligibility unable to be verified.
- Prevent discount stacking unless permitted by admin.

### 3.2.9 <Online Review Scraping + Integration>

3.2.9.1 Introduction
The system shall retrieve and display critic ratings and quotes from external review sources (ie. Rotten Tomatoes, IMDB).

3.2.9.2 Inputs
- Movie title query
- External API/scraping request from review sources (ie. IMDB, Rotten Tomatoes) .

3.2.9.3 Processing
- Query external review platforms.
- Retrieve ratings and critic quotes.
- Refresh data every chosen interval (ie. 24 hrs).

3.2.9.4 Outputs

- Display ratings from reviews alongside movie listings.
- Display selected critic quotes.

3.2.9.5 Error Handling

- Display placeholder message if review retrieval fails.
- Log external API/scraping failures without affecting ticket sale mechanism.

### 3.2.10 <Multilingual Support>

3.2.10.1 Introduction

The system shall support the following languages: English, Spanish, and Swedish.

3.2.10.2 Inputs

- User language selection.
- Account language preference chosen upon sign-up.

3.2.10.3 Processing

- Load interface text based on selected language.
- Store language preference in account/session data.

3.2.10.4 Outputs

- Interface displayed in selected language (English, Spanish, Swedish)

3.2.10.5 Error Handling

- Default language set to English if language fails to load.
- Log language load failures.

### 3.2.11 <Loyalty Account Management>

3.2.11.1 Introduction

The system shall support user account registration and loyalty membership functionality.

3.2.11.2 Inputs

- User registration information.
- Payment information.
- Membership purchase request.
- Sign up/Sign in requests.
- CAPTCHA request.

3.2.11.3 Processing

- Store personal information securely.
- Store encrypted payment data.
- Track purchase history and tie to account details.
- Track account loyalty points.
- Allow membership card purchase.
- The system shall restrict concurrent logins to one active device per user account.

3.2.11.4 Outputs

- Account dashboard.
- Profile page.
- Loyalty point balance display.
- Membership confirmation statement.

3.2.11.5 Error Handling

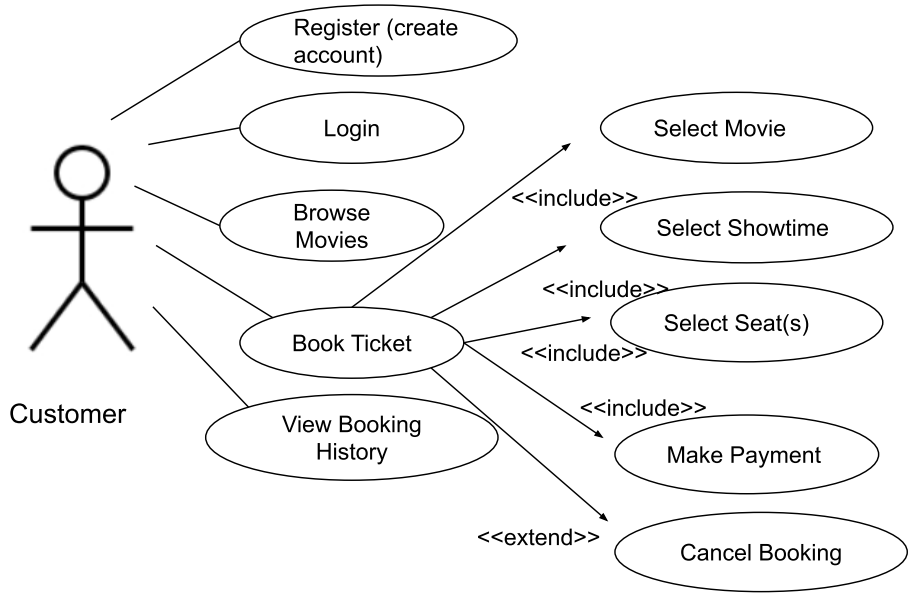
- Prevent duplicate accounts with identical credentials.
- Deny access if authentication fails.

- Log suspicious account activity.

## 3.3 Use Cases

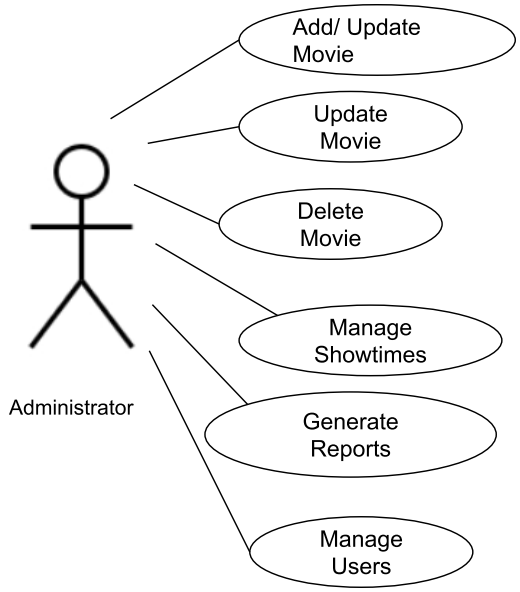
### 3.3.1 Use Case #1 <Customer>:

- Register/ Sign up: Create an account in order to access the system.
- Login: Access an existing account.
- Browse Movies: View the list of the current movies available.
- Book Ticket: Reserve seats for a selected movie, including the following steps:
    - Select Movie <<include>>: Choose a movie to watch.
    - Select Showtime <<include>>: Choose a showtime for the selected movie.
    - Select Seat(s) <<include>>: Choose preferred row and seat number/
    - Make Payment <<include>>: Complete the booking transaction through/
- View Booking History: List of the items in the cart/ the movie times selected.
- Cancel Booking <<extends>>: Cancel any purchased ticket, at least 1-2 hrs before showtime. This is optional and only occurs under certain conditions.



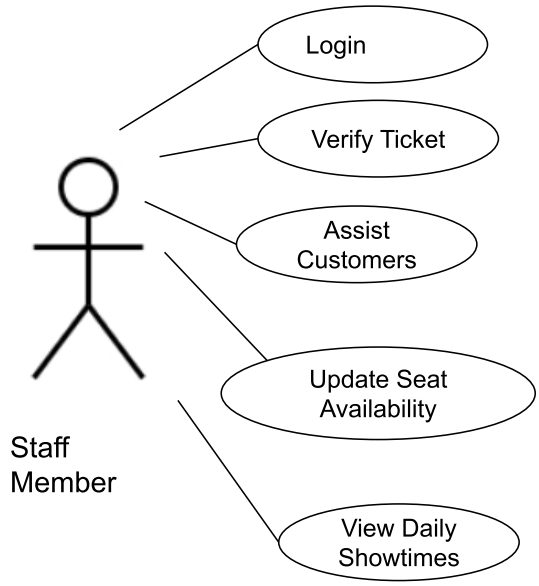

### 3.3.2 Use Case #2 <Administrator>:

- Add Movie: Add a new movie to the system, including the movie details (title, duration, genre, and rating)
- Update Movie: Edit movie details, such as scheduled times and description
- Delete Movie: Remove a movie from the system that is no longer available.
- Manage Showtimes: Schedule, update, or remove showtimes for movies.
- Generate Reports: Ticket sales, booking history, and user activity.
- Manage Users: Create, update, or remove staff/ administrator accounts as needed.




**3.3.3 Use Case #3 <Staff Member>:**

- Login: Login to access staff functions.
- Verify Ticket: Check confirmation code, in person, to validate booking.
- Assist Customers: Help customers with booking issues, cancellations, and general questions.
- Update Seat Availability: Adjust seat availability based on bookings and cancellations.
- View Daily Showtimes: View the scheduled movies for the day



## 3.4 Classes / Objects

3.4.1 <User>
3.4.1.1 Attributes
UserID → String
Name → String
Email → String
Password → String
User Type → Enum {Customer, Staff, Admin}
Loyalty Points → Int
Account Active → Boolean

3.4.1.2 Functions
- Login
- Logout
- Purchase tickets
- Submit feedback
- View tickets
- View loyalty points

### 3.4.2 <Ticket>

3.4.2.1 Attributes
Ticket ID → String
User ID → String
Showtime ID → String
Seat Number → String
Ticket Type → Enum {Regular, Deluxe}
Price → Float
Purchase Time → DateTime
Status → Enum {Active, Cancelled, Taken}

3.4.2.2 Functions
- Ticket purchase
- Ticket validation
- Ticket cancellation
- Ticket transfer

### 3.4.3 <Movie>

3.4.3.1 Attributes
Showtime ID → String
Movie Title → String
Theatre ID → String
Start time → DateTime
End Time → DateTime
Seat Map → Map <String, Boolean> (seat number, T/F isBooked)

Price → Float
Available discounts → Enum {Senior,Veteran,Student}

3.4.3.2 Functions
- Check availability of seat
- Reserve seat
- Book seat
- Apply discount rate

**3.4.4 <Administrator>**

Inherits from User class

3.4.4.1 Attributes
Showtime ID → String
Movie Title → String
Theatre ID → String
Start time → DateTime
End Time → DateTime
Seat Map → Map <String, Boolean> (seat number, T/F isBooked)
Price → Float
Available discounts → Enum {Senior,Veteran,Student}

3.4.4.2 Functions
- Update Movie
- Update Showtime
- Update Prices
- Override ticket
- Manage Users
- View analytics
- View logs

**3.4.5 <Payment>**

3.4.5.1 Attributes
Payment ID → String
User ID→ String
Amount → Float
Payment Method → Enum {CreditCard, PayPal, Bitcoin}

3.4.5.2 Functions
- Process payment
- Refund payment
- Link to external payment service

### 3.4.6 <Feedback>

3.4.6.1 Attributes
Feedback ID → String
User ID→ String
Ticket ID → String
Rating → Enum {1,2,3,4,5,6,7,8,9,10}
Comment → String

3.4.6.2 Functions
- Submit Feedback
- Get average rating

### 3.4.7 <Theatre>

3.4.7.1 Attributes
ID → String
Name → String
Address → String
City → String
State → String
Zipcode → String

3.4.7.2 Functions
- Update location

## 3.5 Non-Functional Requirements

*Non-functional requirements may exist for the following attributes. Often these requirements must be achieved at a system-wide level rather than at a unit level. State the requirements in the following sections in measurable terms (e.g., 95% of transaction shall be processed in less than a second, system downtime may not exceed 1 minute per day, > 30 day MTBF value, etc).*

### 3.5.1 Performance

- The system shall support at least 1000 concurrent users without performance degradation.
- 95% of page loads shall complete within 2 seconds under normal operating conditions.
- Ticket purchase transactions shall complete within 5 seconds, excluding any third party processing delays.
- Seat availability shall update in real time in accordance with confirmed purchase (within 1 second).
- System shall implement a queuing mechanism for high-demand showings to prevent server overload.

### 3.5.2 Reliability

- The system shall maintain accurate seat inventory with 0 duplicate bookings.
- Confirmed ticket transactions shall not be lost in the event of a server failure.
- Daily logs of all ticket purchases shall be maintained for administrative review.
- Transaction failure rate must fall below 1%.

### 3.5.3 Availability

- The system shall maintain 99.99% time up during the theater operating hours.
- Scheduled maintenance shall not exceed 2 hours per month.
- In the event of a system outage, service restoration shall occur within 15 minutes of the outage.

### 3.5.4 Security

- All user data shall be transmitted using HTTPS encryption.
- Passwords shall be stored using hashed encryption methods.
- Each ticket shall contain a unique and non replicable identification number to prevent duplication or resale.
- The system shall restrict concurrent logins to one device per account.
- CAPTCHA verification shall be implemented to prevent bot automated purchase.
- Payment information shall be encrypted and not stored in plain text formatting.
- User sessions shall automatically expire after 15 minutes of inactivity.
- The user ticket checkout portal must automatically expire after 5 minutes.

### 3.5.5 Maintainability

- The system shall follow object oriented design principles, which will allow the individual components of the application (ie. ticketing, payment, feedback, emailing, etc.) to be updated independently.
- All the code shall be documented with inline comments and summarizations for maintenance and update.
- Error logs shall be detailed and provide valuable insight and information for administrators and developers to diagnose and resolve issues within a 24 hour period.
- System updates and maintenance shall be planned to minimize downtime, not exceeding 30 minutes/update cycle.
- The system shall allow administrators to adjust movie listings, showtimes, prices, etc. with ease and dynamic nature.

### 3.5.6 Portability

- The system shall operate entirely in a supported web browser, without requiring installation of an application.
- The system shall function correctly on the latest few versions of modern browsers (ie. Chrome, Safari, Edge, etc.)
- The system shall support both desktop and mobile browsers (cross-device functionality)
- The system shall be compatible with standard devices used by cinema staff for ticket validation (ie. QR code scanners, tablets, mobile phones, etc.)

# 4. Software Design Specification

## 4.1 Software Architecture Design Overview

*The Movie Ticketing System will follow a layered web application architecture, which separates the client/presentation layer (User Interface), an application/backend layer, and a data/database layer. This architecture allows the user interface, system logic and the data management of the system to perform as individual components, which improves scalability,*

*maintainability and reliability of the system. The system allows users to browse movies, select showtimes, choose seats, complete payments, and receive digital tickets.*

*The presentation layer, presented to the client includes a login and sign up page, as well as a movie-selection page with showtimes and prices presented upfront to the client. When a show is selected, seat selection is presented to the client in the case of premium theatre status. Data for the presentation page is regularly fetched and updated through database access.*

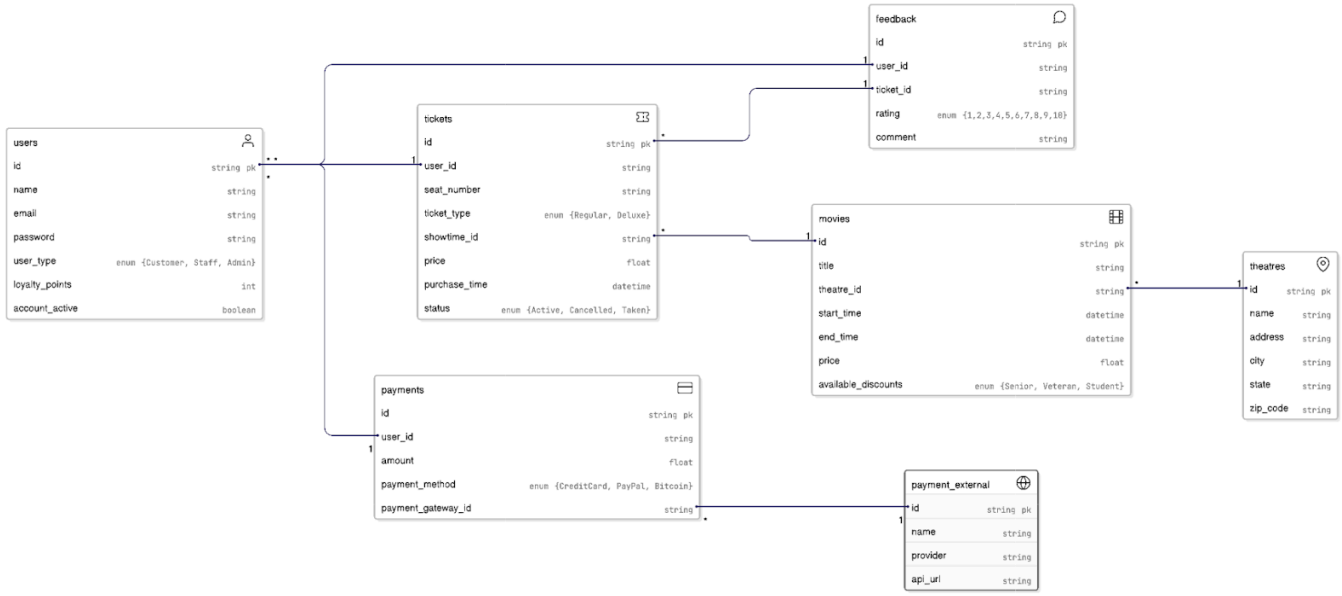
*The application/backend layer contains the core logic behind the Movie Ticketing system. This layer processes user requests which are received from the presentation layer and executes system functionality including authentication, ticket booking, seat reservation, payment processing, and administrative operations for the user. The backend enforces the system requirements and regulations such as ticket purchase limits, seat availability validation, checkout timers, prevention of duplicate bookings, etc. It also manages user sessions, coordinates communication with external services, and coordinates with the system database.*

*The database layer of the system consists of a centralized database, which is responsible for storing and managing the system data. This includes movie listings, theater locations, seat maps, user accounts, ticket purchases, feedback submissions, transaction history, etc. The database works to ensure consistent ticket inventory as well as seat availability, and works to update this in real time when tickets are purchased or cancelled. System logs are also stored in the database to support administrative analytics and system monitoring.*

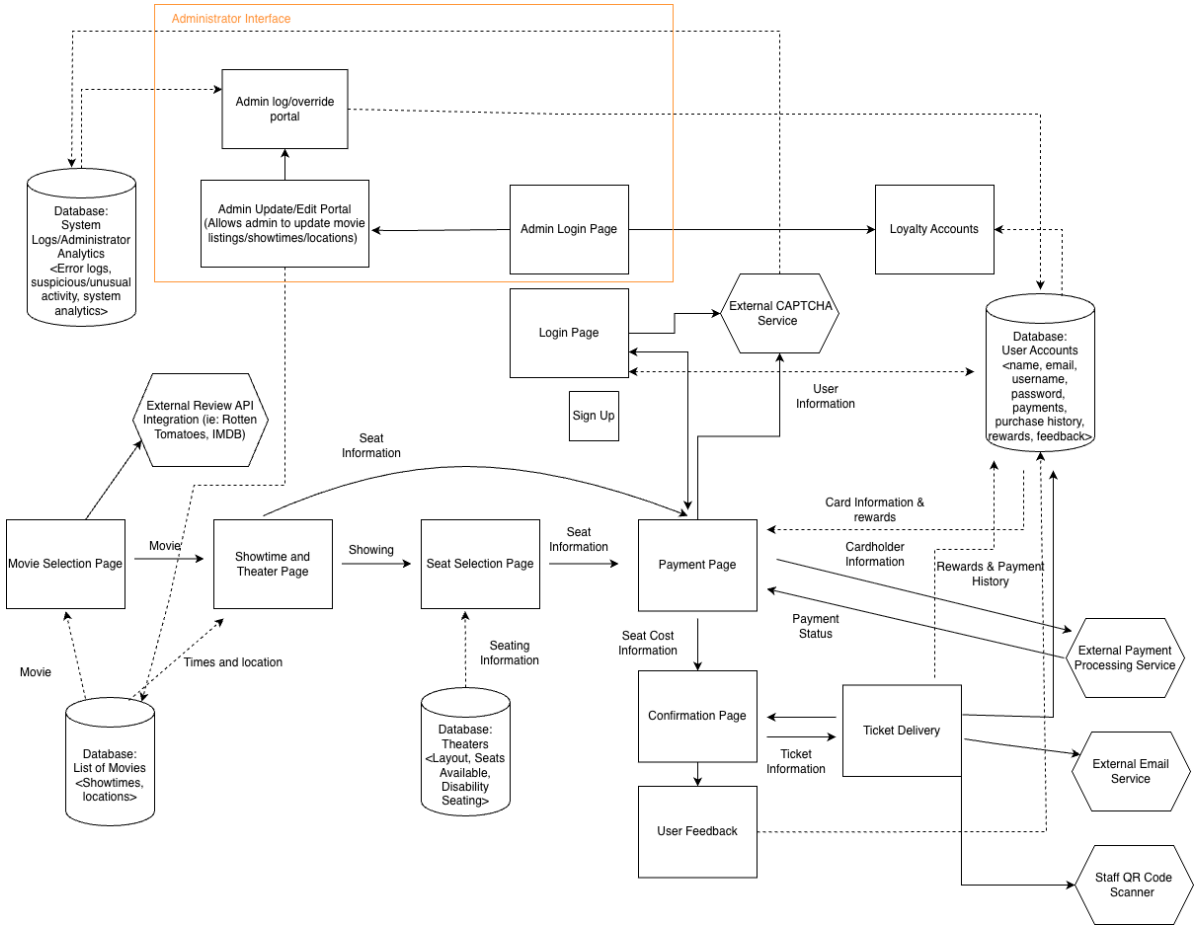
*External services are also integrated into the system to support key functions such as payment processing, email delivery, reviews, etc. A payment processing service handles financial transactions (e.g. credit card purchases, online payments). An email delivery service is used to send confirmations, email receipts, and digital tickets. A CAPTCHA service handles the service to verify user identity and bot prevention.*

*This layered architecture enables the Movie Ticketing System to support high user traffic, maintain consistent availability, and allow future system updates without requiring major system changes.*

## 4.2 UML Class Diagram:




## 4.3 Software Architecture Diagram



## 4.4 Software Architecture Description

The Movie Ticketing System is a web-based platform that guides users through the process of buying movie tickets with the help of databases, external services, and user interface elements. The architecture improves system structure, scalability, and maintainability by dividing the system so that each component carries out a distinct role. The purpose of the architecture is to enable communication between every system page and the relevant database or outside service provider. The software architecture diagram shows the interactions of the user interface pages with internal databases and external services to finish the ticket purchase process. Overall, it is designed to make it possible for customers to quickly and effectively choose movies, pick seats, finish paying, and receive their tickets.

### 4.4.1 Interfaces

The **User Interface** represents the web pages that allow both customers and administrators to interact with the system. These include:

- **Movie Selection Page**: Main user interaction, users can browse available movies at certain theater locations. Displays movie titles with summaries, showtimes, and theater locations.
- **Showtime and Theater Page**: Displays available movie showtimes and theater locations for the selected movie.
- **Seat Selection Page**: Displays seating layout inside the selected theater room allowing users to choose their preferred seat(s).
- **Payment Page**: Total cost of the selected tickets is calculated. May prompt users to log in or create an account through the Login Page, if they have not already done so.
- **Login Page**: Users with existing accounts will be prompted to type in their username and password, otherwise they must sign up with an email and create an account.
- **Confirmation Page**: Displays the final ticket details, including movie title, showtime, theater location, selected seats, and the total cost of purchase.
- **User Feedback**: Gives users a chance to submit feedback about their experience.
- **Ticket Delivery**: Sends the purchased ticket to the customer via email.

The system includes an **Administrator Interface** for managing and updating the platform. Administrators must login through the **Admin Login Page**, giving them access to the following:

- **Admin update/ Edit Portal**: Allows admin to update movie listings, showtimes, and locations
- **Admin Log/Override Portal**: Record administrative activity, monitors system performance, and allows authorized overrides when necessary

### 4.4.2 Databases

The Movie Ticketing System uses several databases to store and manage information needed during the ticket purchasing process.

- **List of Movies Database**: Stores movie titles, showtimes, and theater locations. Displayed in the Movie Selection Page and Showtime and Theater Page when users browse through available movies.

- **Theaters Database**: Stores theater layouts, seat availability, and accessibility seating information. Displayed on Seat Selection Page to show the seating arrangement and ensure users can only select seats that are available during the time of purchase.
- **User Accounts Database**: Stores customer account information such as names, email addresses, usernames, passwords, saved payment methods, purchase history, rewards information, and user feedback. This database supports the Login/ Sign Up pages and keeps track of purchases after a transaction is completed.
- **System Logs and Admin Analytics Database**: Stores administrative actions, system errors, suspicious activity, and other analytics data. This helps administrators monitor system performance, maintain security, and track overall usage.

**4.4.3 External Services**

The system relies on several external services to provide additional features, security, and communications.

- **External CAPTCHA Service**: Used during the login and sign in process to prevent automated attacks and protect user account security.
- **External Review API**: Gives movie ratings and reviews that help users learn more about the films they are considering.
- **External Payment Processing**: Securely handles cardholder information and returns a status indication whether the transaction was approved or declined.
- **External Email Service**: Ticket information is sent to the Ticket Delivery which works with this service to send digital tickets to the user.
- **QR Code Scanner**: Allows theater staff to scan digital tickets when customers arrive and verify their entry.

**4.4.4 Data Management Strategy**

The Movie Ticketing System uses a centralized relational database (SQL-based system) that manages all application data. We chose an SQL database because the system depends on the accuracy and consistency of its data, most important for critical operations such as seat availability, ticket purchases, and seat reservations to ensure consistency across the entire system.

**Database Choice:**

We are using one main centralized SQL database (ie: MySQL/PostgreSQL). We prefer this over multiple databases since a singular database allows for easier management and system consistency.

A relational database works for this system due to the fact that:

- It keeps relationships between data separate and clear (users, movies, tickets, etc.).
- It supports transactions when multiple people buy tickets at the same time.
- It aids in preventing duplicate seat bookings and other issues of duplication.

**Data Organization:**

The database is split into a few main tables:

- **User Accounts:** Stores account information such as name, email, password (hashed), role (ie: customer, staff, admin), loyalty points.
- **Movies:** Stores movie details such as title, description, genre, rating, duration.
- **Theatres:** Stores theater information such as location, seating layout, etc.
- **Showtimes:** Stores connective information connecting movies to theatres and showtimes.
- **Seats/Seat Map:** Stores available and non-available seats, reservation details and bookings.
- **Tickets:** Stores ticket details such as seat number, showtime, price, and status.
- **Payments:** Stores payment information and receipts (Actual transactions and sensitive information are handled externally).
- **Feedback:** Stores user ratings, comments, and other feedback formats.
- **Logs:** Stores system and admin activity for debugging and monitoring.

**Data Lifecycle:**

When a user books a ticket, this is the flow:

1. The user selects a movie and a showtime.
2. The system pulls data from Movie and Showtime tables.
3. Available seats are loaded from the database.
4. When a user selects a seat, it gets temporarily reserved.
5. A 5 minute timer is started during checkout.
6. If payment succeeds a ticket is created and saved in the Tickets table. Receipt information is saved in the Payment table.
7. Seat status is updated to ‘booked’.
8. A confirmation message is sent through email, consisting of a ticket/QR code using an external service provider.
9. If the user doesn’t finish payment in time, the seat is released back to ‘available’.

**Handling Multiple Users at the Same Time:**

Since a lot of people have access to the system at once, this is how it is handled:

- Utilize database transactions so ticket purchases either fully complete or don’t go through.
- Seats are temporarily locked during selection.
- Locks expire if the user doesn’t finish checkout in time.
- If something fails, the system rolls back changes to avoid data mismatches.

**Why we chose this option:**

Utilizing a NoSQL database or splitting the database into multiple services were other considerations, but we decided to go with SQL instead since:

- It is harder to enforce and define strict relationships between components (ie: seats, tickets, movies)

- NoSQL doesn't handle transactions as well as SQL.
- It could lead to inconsistencies during high-demand ticket sales.
- Splitting the database would be overkill and unnecessarily complicate the system more than needed for this project.
- A single database is more simple and consistent, making it overall most efficient and reliable. A single SQL database allows for a balance of correctness and scalability.

**4.4.5 Tradeoff Discussion:**

There were a few design decisions that were made to balance simplicity, performance, and scalability. The main choice was to utilize a single centralized SQL database, instead of multiple databases or a distributed database system. This was chosen due to the consistency of the data being kept and the simplicity of data management. Additionally, it helps prevent issues such as double booking of seats. The tradeoff is that a single database can become a limitation if the system grows to a very large scale, but in the case of this project and expected use case it serves its purpose.

A relational database was also chosen by us instead of NoSQL since SQL works better for the system due to its high levels of structure and connectivity. It connects the components (ie: users, tickets, movies, seats, etc.) efficiently. The tradeoff is that SQL is less flexible when it comes to distributed scaling. The benefits of consistency outweighed this though, and therefore we decided to keep the relational database system.

Additionally, the utilization of a layered architecture including the components of a frontend, backend and database instead of another architectural form (ie: microservices) was a design choice with tradeoffs. The form of layered architecture was chosen due to its implementation simplicity, as the division of each layer allows maintenance and build to be much more efficient and encapsulated, making it easier to build and debug. The tradeoff is the scalability of microservices architecture, as well as better independently offered services. Unfortunately, microservices would cause a lot of issues when it came to its complexity and make coordination as well as build and maintenance between services to be much more difficult.

Another design choice was the utilization of external payment, email and CAPTCHA services instead of building our own systems. This has the benefit of development time and cost reduction, as well as security improvements since the external services are well maintained and tested already. A tradeoff is that the system has a dependency on third-party services, which could be harmful to our system in a situation where one is down or must be maintained. The benefit of a pre-made and well tested system handling complex issues for us outweighed the tradeoff, allowing us to choose to continue utilizing external systems for these certain tasks.

Overall, the design choices were made to focus primarily on reliability, accuracy, and simplicity of the system rather than extensive scalability and flexibility, due to the specific requirements of this system as well as the design and user-base.

## 4.5 Development Plan and Timeline

### 4.5.1 Partitioning of Tasks

Development approach
The team will use an iterative milestone approach. Each milestone produces a working build that can be demonstrated end to end, even if some features are still simplified (for example, mock payment first, then real integration). Work is prioritized around the main customer flow shown in the architecture: browse movies, choose showtime, select seats, pay, receive confirmation and ticket delivery, then staff validation and admin management.

Task breakdown by work packages

1. Planning and project setup

- Confirm and lock functional and non functional requirements (concurrency, bot protection, purchase constraints, multilingual, loyalty, admin, staff validation).
- Define acceptance criteria and test cases for each major feature.
- Create initial UI wireframes and finalize database schema.
  Deliverable: requirements checklist, acceptance criteria, architecture alignment, schema draft.

2. Database and core backend foundations

- Implement database tables and relationships for Users, Tickets, Movies, Theatres, Payments, Feedback, Logs.
- Implement core API endpoints for browsing movies, retrieving showtimes, seat availability, and ticket inventory.
- Implement session management and authentication for customer, staff, and administrator roles.
  Deliverable: backend skeleton and database running with seed data.

3. Customer purchase flow (core system functionality)

- Movie Selection Page and Showtime and Theater Page.
- Seat Selection Page for deluxe theatres (reserved seating) and quantity only tickets for regular theatres (general admission).
- Enforce ticket purchase constraints (max 20 tickets, two week booking window, 10 minutes after showtime begins).
- Implement 5 minute checkout timer and automatic release of reserved seats on expiration.
  Deliverable: working demo from browse to seat selection with live availability updates.

4. Bot protection and security hardening

- Integrate CAPTCHA at login and checkout.
- Implement basic rate limiting and suspicious activity flagging.
- Store passwords using hashing and enforce session expiration rules.
  Deliverable: bot protection and security requirements met in the main flow.

5. Payment and ticket delivery integration

- Integrate payment processing (sandbox or mock first, then refine).
- Generate ticket IDs and QR codes.
- Email delivery of receipt and QR code ticket to customer.
- Store purchase history and rewards information in User Accounts Database.
  Deliverable: purchase to confirmation to email delivery works reliably.

6. Staff validation features

- Staff login and access to ticket validation functions.
- Ticket lookup or QR scan workflow to confirm status (Active, Cancelled, Taken).
- Display showtime and seat details quickly for staff.
  Deliverable: staff can validate tickets and mark entry.

7. Administrator mode features

- Admin login and admin update and edit portal.
- Manage movies, showtimes, theatres, seat inventory rules, and pricing.
- Manage users (block and unblock, suspend and unsuspend) and view basic logs or analytics.
  Deliverable: admin can update listings and monitor system activity.

8. System testing, performance checks, and final documentation

- End to end testing for critical flows: browse, purchase, timer expiration, cancellation policy, ticket validation, admin updates.
- Concurrency testing scenario based on requirement of 1000 concurrent users.
- Finalize documentation: setup instructions, assumptions, known limitations, and demo script.
  Deliverable: final stable build and submission package.

Proposed timeline (Deadline is May 7, 2026)

Week 1 to Week 2: Planning and setup

- Requirements lock, wireframes, schema, project repo setup.

Week 3 to Week 5: Core backend and customer flow

- Database, APIs, browse and showtime selection, seat selection, purchase constraints, checkout timer.

Week 6 to Week 7: Payment and ticket delivery

- Payment integration (sandbox or mock), QR generation, email delivery, purchase history.

Week 8 to Week 9: Staff and admin modules

- Staff ticket validation workflow, admin management features.

Week 10 to Week 11: Quality and performance

- Security hardening, bot protection tuning, concurrency testing, bug fixes.

Week 12: Final packaging and demo prep

- Final documentation, polishing UI text and multilingual strings, final verification of requirements.

### 4.5.2 Team Responsibilities

All team members

- Participate in requirements review, code reviews, and integration testing.
- Maintain a shared test plan and end to end test cases.
- Contribute to documentation and final demo preparation.

Michelle Khaliqi

- Lead customer facing UI implementation:
    - Movie Selection Page, Showtime and Theater Page, Seat Selection Page, Payment Page UI, Confirmation Page UI.
- Implement multilingual UI text support (English, Spanish, Swedish) and usability checks.
- Coordinate UI integration testing with backend endpoints.

Nandana Manu

- Lead backend and database implementation:
    - Database schema, core APIs, seat availability updates, reservation and release logic.
- Implement authentication and session management across user roles.
- Lead concurrency and data integrity controls to prevent duplicate bookings.

Katarina Cedolin

- Lead integrations and operational features:

    - Payment processing integration (sandbox or mock) and email ticket delivery workflow.
- Implement staff ticket validation flow (QR scan or lookup and status updates).
- Implement administrator mode features for managing movies, showtimes, pricing, and user controls.

## 4.6 Test Plan

| SDS: Test Plan Link | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **TestCaseId** | **Component** | **Priority** | **Description/Test Summary** | **Pre-requisites** | **Test Steps** | **Expected Result** | **Actual Result** | **Test Executed By** |
| Test_01_ValidUserLoginFunction | User Class | P0 | Verify that the Login() method authenticates a valid user, generates a session token, and logs the event. | Database contains user record, including:<br>- Email: testuser@example.com.<br>- Password hash matches input password.<br>- Account is active<br>CAPTCHA challenge passed<br>No active sessions exist for this user (System restriction on concurrent login). | 1. Call Login(email, password) with valid credentials).<br>2. System retrieves user record and compares hashed password.<br>3. System generates a session token. | - Function returns TRUE, session token is generated.<br>- Session token is created and stored in database.<br>- User directed to dashboard. | Login function returns TRUE, session token generated. | Nandana Manu |
| Test_02_Browse Movies | Movie Browsing | P0 | Verify user can view all movies that are available | Movies exist in database | 1. Navigate to homepage<br>2. View list of movies | List of movies with title, showtime, and description is displayed | List of movies with title, showtime, and description is displayed | Michelle Khaliqi |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| Test_03_SeatSelection | Seat Selection | P0 | Verify user can view all seats that are available | User selected a movie and showtime | 1. Navigate to seat selection page.<br>2. Select an available seat.<br>3. Confirm and update selection page and record in database. | Selected seat is reserved and unavailable to others | Selected seat was successfully reserved and updated seat selection page to unavailable for others | Michelle Khaliqi |
| Test_04_Process Payment | Payment Module | P0 | Verify payment is processed successfully with the valid information | User has selected a movie, showtime, and seat. Ready to proceed and pay. | 1. Navigate to payment method page.<br>2. Enter valid payment details (card number, expiration date, CVV).<br>3. Process and confirm payment. | Payment is approved and returns successful status/ confirmation page | Payment is successfully processed, booking confirmation page is displayed with QR code and booking ID | Michelle Khaliqi |
| Test_05_InvalidPayment | Payment Module | P0 | Verify payment was not processed successfully when invalid payment details are entered | User has selected a movie, showtime, and seat. Ready to proceed and pay. | 1. Navigate to payment method page.<br>2. Enter invalid payment details (card number, expiration date, CVV).<br>3. Process and reject payment. | - Payment is declined and user stays on the payment page.<br>- Error Message displays. | Payment is declined, Error message is displayed. No booking is created. | Michelle Khaliqi |
| Test_06_InvalidUserLoginFunction | User Class | P0 | Ensure login fails when password credentials are incorrect. | User exists and user's CAPTCHA challenge has passed. | 1. Call Login(email, wrongPassword).<br>2. System compares hashed passwords. | - Login returns FALSE.<br>- No session token is generated.<br>- Error Message displays. | Login function returns FALSE, session token is not generated, error message displays. | Nandana Manu |